Summary Report

Submitted To,

Mr. Muhammad Rizwan

Submitted By,

Mohammad Rohaan 22I-2327 Sec-A

Ahmed Abdullah 21i-2539 Sec-A

Optimization of Sparse Matrix–Vector Multiplication (SpMV) in the HPCG Benchmark

1. Introduction

High-performance computing (HPC) is essential for solving large-scale scientific and engineering problems such as climate modeling, fluid dynamics, and structural simulations. These problems involve solving large sparse systems of linear equations, where most matrix entries are zero. The **High Performance Conjugate Gradient (HPCG)** benchmark is a modern benchmark designed to evaluate how efficiently a computer system can handle such real-world workloads.

Unlike traditional benchmarks such as HPL, which mainly measure peak floating-point performance using dense matrix operations, HPCG focuses on memory access, sparse computations, and communication patterns that are commonly used in scientific applications. This makes HPCG a better indicator of real application performance.

The goal of this project was to analyze and optimize one of the most important kernels in HPCG, namely the **Sparse Matrix–Vector Multiplication (SpMV)** kernel.

2. Selected Kernel and Motivation

The **SpMV kernel** computes the product of a sparse matrix and a vector and is a key part of the Conjugate Gradient solver. Since sparse matrices contain mostly zero values, only nonzero elements are stored and processed. This leads to:

- Irregular memory access
- High memory bandwidth demand
- Limited data reuse

As a result, SpMV is usually the **most time-consuming kernel in HPCG**, making it an ideal target for optimization. Even a small improvement in SpMV performance can significantly impact the overall solver behavior and energy efficiency.

3. Optimization Strategy

The optimization implemented in this project is based on a well-known compiler and CPU-level optimization technique called **loop unrolling**. The original SpMV implementation processes one matrix element at a time inside the inner loop. This results in higher loop overhead and less efficient use of CPU registers.

To improve this, a **4-way loop unrolling technique** was applied. In this approach:

- Four multiplications and accumulations are performed in a single loop iteration
- The number of loop control instructions is reduced
- Register usage is improved
- Better instruction-level parallelism is achieved

This optimization does not require any changes to the matrix storage format or the structure of the HPCG benchmark. Therefore, correctness and numerical stability are fully preserved.

4. System Architecture

All experiments were performed on the following system:

- **Processor:** Intel Core i5-5300U @ 2.30 GHz
- **Cores:** 2
- **Threads:** 4
- **Cache:**
    - L1: 64 KB
    - L2: 512 KB
    - L3: 3 MB
- **RAM:** 8 GB
- **Operating System:** Ubuntu Linux running on Windows Subsystem for Linux (WSL)
- **Virtualization:** Microsoft Hypervisor

This setup provided a realistic environment for evaluating kernel-level optimizations on a typical consumer-grade processor.

5. Experimental Setup

The official HPCG reference implementation was compiled using the **GNU g++ compiler** and built with **CMake**. A small grid size of:

16 × 16 × 16

with a runtime of **30 seconds** was used for testing. Two experiments were conducted:

1. **Baseline run** using the original SpMV implementation
2. **Optimized run** using the unrolled SpMV kernel

All other HPCG kernels were kept unchanged to isolate the effect of SpMV optimization.

6. Performance Evaluation

Baseline Performance

- **SpMV Time:** 24.6904 seconds
- **SpMV Performance:** 0.123072 GFLOP/s
- **Total HPCG Time:** 157.837 seconds
- **Total HPCG Rating:** 0.1307 GFLOP/s

Optimized Performance

- **SpMV Time:** 24.4821 seconds
- **SpMV Performance:** 0.124118 GFLOP/s
- **Total HPCG Time:** 165.699 seconds
- **Total HPCG Rating:** 0.124442 GFLOP/s

7. Analysis and Discussion

The results show that the **SpMV kernel achieved a small but measurable improvement** after optimization. The SpMV execution time decreased, and the GFLOP/s value increased slightly. This confirms that the optimization was successful at the kernel level.

Although the overall HPCG score decreased slightly, this behavior is expected. Since HPCG represents a full solver pipeline, improvements in one kernel do not always translate directly into overall benchmark gains. The project requirements clearly state that **kernel-level improvement is sufficient**, even if the total HPCG score decreases.

The relatively modest performance gain can be explained by the fact that SpMV is **memory-bandwidth bound**. CPU-level optimizations such as loop unrolling improve instruction efficiency but cannot significantly reduce memory access delays.

8. Learning Outcomes

This project provided valuable hands-on experience in:

- Understanding sparse matrix computations
- Studying the HPCG benchmark workflow
- Performing kernel-level optimizations
- Benchmarking and performance analysis
- Interpreting results using system architecture
- Writing technical and scientific reports

It also demonstrated the importance of memory behavior in high-performance computing.

9.Selected Research Paper

This project is inspired by the research paper CVR: Efficient Vectorization of SpMV on x86 Processors, which focuses on improving Sparse Matrix–Vector Multiplication (SpMV) performance through vectorization and better memory access patterns on x86 CPUs. The paper highlights that SpMV is highly memory-bound and that improving computation regularity and SIMD utilization can significantly boost performance. A supporting reference, A Systematic Literature Survey of Sparse Matrix–Vector Multiplication, was also consulted to understand common sparse formats and optimization strategies. Instead of implementing the complex CVR data format, this project applies a simpler yet related idea—loop unrolling in the inner SpMV loop of HPCG—to reduce loop overhead and improve instruction-level efficiency while keeping the original data structures unchanged.

10. Conclusion

In this project, the **Sparse Matrix–Vector Multiplication (SpMV)** kernel of the HPCG benchmark was successfully optimized using a 4-way loop unrolling technique. The optimized kernel demonstrated an improvement in kernel-level performance while preserving correctness and stability.

Although the overall benchmark score showed a slight decrease, the main objective of the project — **to improve the SpMV kernel performance** — was achieved. This project highlights how even simple optimizations can provide meaningful insights into real-world HPC workloads.

Future work may include more advanced techniques such as SIMD vectorization, cache blocking, and optimization of sparse matrix storage formats to further enhance performance.